

TABLE 1. The test activities, with methods, tools, and usage schemes.

Activity	Test methodology	Defect types	Blind spots	Tools	Effort
Static analysis	• Method to detect defects at early stage development of software unit • Design inspection for software unit • Review of end user functionality, e.g., use cases • Validation of functional requirements • Review of descriptions for database, interfaces, input/output, hardware, software and network • Techniques: code reviews, walkthroughs, inspections, and flow analysis	• Design defects • Insufficient functional requirements • Inconsistent design and architecture specifications • Memory and buffer overflows • Code complexity • Code, syntax, and standardized labeling errors	• Dynamic behaviors of the code execution • Processor utilization • The number of reported deficiencies makes most engineers turn off warnings, thus creating massive blind spots	Lint, Soot, CheckStyle, SourceMeter, KlocWork, Parasoft C/C++ with CERT C rules support, CodeSonar C/C++, Imagix 4D, Testwell CMT++ CTC++, VectorCast	Medium, slightly automated
Unit testing	• Method to test a software unit • White-box testing • Observes TDD • Tests to verify functional (software) requirement “input X generates Y” • Techniques: equivalence classes analysis for high-variance classes, boundary value analysis, code coverage testing (statement, branch and modified condition/decision coverage)	• Potential bugs in the code base • Missing functionality in comparison to the requirements • Buffer overflows • Exception control and process utilization	• Integration errors • System errors • Functionality across multiple software units • Intrinsic dependencies among software units • Quality requirements • Unclear coverage schemes	Junit, PHPUnit, NUnit, UnitTest ++, pytest, , Testwell CMT++ and CTC++, CANTATA, ADATest 95, Mockito, TestNG, VectorCast, CANoe4SW	Low, highly automated
Function testing (software)	• Method to test SW functionality based on System requirements • Black-Box Testing of SW Application • Analysis of defined, expected, and recorded input and output values • Test Case definition based on Functional and Business Requirements • Techniques: Smoke Testing, Sanity Testing, Interface Testing, Acceptance Testing	• SW Application’s main functions • Functionalities of the user interface, APIs, I/O’s and SW Database • Operating Limitations in proprietary application specific SW Units • Functionality spanned across multiple SW units	• Quality Requirements • Hardware deficiency cannot be captured • Code based vulnerabilities cannot be identified • AI and adaptive systems are not covered	TestComplete, UFT, Parasoft Suite, Selenium, QTP, SoapUI, Watir, Wireshark, Tcpdump, LoadRunner, JMeter, VectorCast, VT System	Medium, highly automated
Function testing (component)	• Method to test software component, based on system requirements • Black-box testing of software application • Tests a slice of functionality of the whole system • Validation against test requirements and use cases • Analogous to static analysis and unit testing	• Errors in functions (stubs) and calling functions (drivers) • Design defects • Errors in software component tasks • Bugs and inaccuracies in the test plan and specifications	• Quality requirements • Design conditions • Design preferences • Structural limitations of the component under test	CANalyzer, CANoe, vTESTstudio, VectorCAST, JUnit, PHPUnit, NUnit, UnitTest ++, Pytest Selenium, QTP, SoapUI, DSO, software-defined radio, Locust, NeoLoad	Medium, highly automated

Activity	Test methodology	Defect types	Blind spots	Tools	Effort
Integration test	- Method to test a software application i.e., the combination of multiple software units - Approach: White box, black box, and gray box - Techniques: Big bang, incremental, top down, and bottom up - Priority testing of critical modules (top-down) - Integration testing supports evaluation of the software application against requirement changes	- Integration errors from different software units - Functionality across multiple software units - Intrinsic dependencies between software units - Faults at external interfaces and input/output ports - Design flaws can be identified - Quality requirements	- Source of specific defect - Code-based vulnerabilities - Operating limitations in proprietary application-specific software units - Interface links could be missed as number of interfaces increases with number of software units	Cantata, LDRA TBrun, LDRAunit, Citrus, Protractor, Shodan, FitNesse, Hamcrest	High, highly automated
Embedded testing with software in the loop, model in the loop, and hardware in the loop	- Software in the loop for software functionality by simulating the system and hardware - Model in the loop for testing software in a model-based environment - Hardware in the loop by including actual (embedded) components - Validation of different test scenarios by examining the expected and recorded behavior of the input and outputs - Validation of software requirements - With hardware in the loop, real-time simulations are possible	- Functional errors of a software application - Coding errors from implicit code coverage testing (exit criteria for software-in-the-loop testing) - Functional errors of the hardware/device under test as it is “in-the-loop” simulated - Model in the loop: bugs and inaccuracies of the modeled hardware and software - Hardware in the loop: defects of interfaces, exchange messages, and networks - Dependencies between software and hardware	- Source of specific defect - Code-based vulnerabilities - Operating limitations in proprietary application-specific software/hardware units	Software in the loop: CANoe, MATLAB, SimuLink, VSim, RT-LAB, vVIRTUALtarget, IoTIFY, IBM Bluemix (now IBM Cloud), TestView, ScopeView. Model in the loop: PREEvision, MATLAB, SimuLink, vVIRTUALtarget, VISUALCONNX, MICROGen, LOOPY, PLECS, Dymola, ASCET, Scilab Hardware in the loop: VT System, PixHawk, vTESTstudio, DSpace, CANoe, NI Lab View, VeriStand, PXI, CompactRIO, LabVIEW, TestStand	High, highly automated
System test	- Method to validate a software product, i.e., the integration of single or multiple applications - Validation against system requirements - Black-box testing - Derivation of end-to-end testing scenarios for components of software application - Dedicated volume testing, e.g., stress, performance, load - Test for quality requirements, such as availability, accessibility, usability, and so on	- System errors from different software applications - Functional defects of a software product - Defects of communication interfaces, exchange messages, and network functionality - Deficiencies of input/output interfaces - Functionality across multiple software applications - Intrinsic dependencies among software applications - Design flaws can be identified	- Source of specific defect - Code-based vulnerabilities - Operating limitations of proprietary application-specific software applications - AI and adaptive systems are not covered	Quality Center/ALM, SpiraTest, TestMonitor, Eggplant, vFlash, vMDM, vConnect, Postman, Selenium, Wireshark, SoapUI, TestingWhiz, TestComplete, Ranorex	Very high, moderately automated

Activity	Test methodology	Defect types	Blind spots	Tools	Effort
Security testing	• Technique to validate system and security goals • Fuzzing of the electronic control unit bus through signal and frame fuzzing • Validation of security techniques and testing for robustness • White box, black box, and gray box (interface discovery, network discovery, network penetration test, and software penetration test) • Types: signal and frame fuzzing • Combinatorics: sequential, pairwise, combinatorial, and endless	• Vulnerabilities of the SW and system design, e.g., integrity, confidentiality • Communication-based vulnerabilities • Debug interface-based vulnerabilities • Unintended behavior (spoofing identity, tampering with data, repudiation threats, information disclosure, denial of service, and elevation of privileges) • Examination of hidden pathways that may lead to exploiting a vulnerability	• Requirement deficiency cannot be captured • Hardware deficiency cannot be captured • Code-based vulnerabilities cannot be identified • Brute force fuzzing and performance restrictions result in missed defects and vulnerabilities	VectorCAST, CANoe, vTESTStudio, Kali Linux, hardware/software debuggers, controller area network/local interconnect network/Ethernet/FlexRay Interface	Very high, only partially automated for fuzzing
Services testing	• Validate webservices and service-oriented architecture; similar to unit test but on different abstraction levels • Webservice implementation approaches: Simple Object Access Protocol, Representational State Transfer, and Web Services Description Language • Software applications employing webservices for data communication	• Faults in the protocol used to create a webservice • Missing and malformed parameters • Regulation address mismatches	• Webservice availability errors • Interface errors • Immutability of existing and new webservices interface to client applications	SoapUI, Axis2 API, SOAPSonar, SOAtest, TestMaker, Postman, Httpmaster, vREST	High, can be well automated
Cloud testing	• Testing of software applications, software units, and systems by using cloud services. • Software as a service testing is accompanied with testing as a service • Quality assurance testing via cloud tools • Testing cloud tools, architecture, infrastructure, platforms, and so on • Test of cloud services and connectivity • Performance • Cybersecurity • Robustness	• System errors from different software applications • Functional defects of a software product • Defects of communication interfaces, exchange messages, and network functionality • Deficiencies of input/output interfaces • Configuration management helps avoid data loss and data recovery	• Synchronization faults among cloud models • Security and privacy concerns for over-the-air and cloud implementations • Server, storage and network failures impacting testing • Connectivity and availability concerns	SOASTA CloudTest, LoadStorm, BlazeMeter, Amazon Web Services, Jenkins, Xamarin Test Cloud, Nessus, Gatting, Apache JMeter, Selenium	High, automation for service access and performance testing

Activity	Test methodology	Defect types	Blind spots	Tools	Effort
CT	- Validation of system via cognitive test methods - Black-box testing - Methods: AI/ML algorithms, mathematical models - Employing learning to remove redundancy in test case generation (brute force) - Software application accounted for undiscovered scenarios - Accounting software application for corner, misuse, and abuse cases - Scenario-based testing - Works with X-in-the-loop test suite	- Functional defects of a software application in different scenes, situations, and scenarios - Complements defects found in functional, integration, and system testing - Real-time faults detected if approach is used with X-in-the-loop test suite	- Quality requirements - Software fault localization difficult - Hardware fault localization difficult	Requirements engineering databases, such as DOORS, TensorFlow, Apollo Baidu, Autoware, Open pilot, MATLAB, LGSVL, Vires, dSPACE ASM Toolchain, Carla, CarSim, and AirSim	High, highly automated
Qualification testing	- Test product against relevant stakeholder, functional, and business requirements - Result-driven method to ensure fidelity of software to requirements - Pass/fail criteria for qualification assessment - Quality requirements also included - Failure model and effects analysis for qualification - Test of software against contractual qualification levels, e.g., similarity, comparison, goal certification, and life cycle prediction	- Performance features of software against requirements checklist - Potential defects that may lead to failure in qualifying software/hardware - Vulnerabilities of software against stress testing	- Design flaws - Design conditions - Design preferences - Structural limitations - Source of specific defect - Code-based vulnerabilities - Operating limitations of proprietary application-specific software applications	Requirements engineering databases, such as DOORS, PREEvision, PCAN View, PCAN explorer, CS+, S32 Studio, and LT Spice	High, not automated